

Kotlin

DATE: _____

PAGE: _____



(A)

Q-1

is the file extension of source code in

1) Kotlin.

→ .kt

2) Immutable variables declared using - val
Keywords

3) Explain this keyword.

→ this refers to the current instance of the class.

4) Explain Super keyword.

→ Super refers to the parent class and is used to access its properties.

(B)

1) What is Kotlin?

→ Kotlin is a statically typed, cross-platform programming language developed by JetBrains. It is designed to be fully interoperable with Java and runs on the Java Virtual Machine (JVM) making it a popular choice for Android app development. Kotlin offers modern language features like null safety, extension function, and coroutines, which enhance productivity and code quality.

→ it is fully interoperable with Java and is widely used for Android development, web development and Server-side application due to its Concise Syntax, Safety features, and expressive coding style.

Q) Explain program execution architecture of Kotlin.

1) Compilation & Kotlin Source code (.kt files) is compiled into Java bytecode using the Kotlin compiler.

2) Execution & The generated bytecode runs on the Java Virtual Machine which translates it into machine code of execution.

⇒ For Kotlin/Native, the code is compiled directly into native binaries and for Kotlin/JS, it is transpiled into JavaScript for web application.



(C)

1) Explain visibility modifiers of Kotlin.
 → In Kotlin, visibility modifiers determine the accessibility of classes, objects, interfaces, constructors, functions, properties, and their setters. The primary visibility modifiers are:-

1) public & This is the default visibility modifier. The member is accessible from everywhere in the project.

2) Private & The member is only accessible within the file or class it is declared in.

3) protected & The member is accessible within the class it is declared in and any subclasses. It is not available at the top level.

→ Similar to private but also accessible in subclasses.

4) internal & The member is accessible within the same module (a group of files compiled together).

→ Accessible within the same module but not outside it.

→ Accessible only within the same class or file.

2) List operators of Kotlin, explain any one.

1) Arithmetic operators (+, -, *, /, %)

2) Relation " (=, !=, >, <, >=, <=)

3) Logical " (&, &&, ||, !)

4) Bitwise " (shl, shr, ushr, and, or, xor)

5) Assignment operators ($=$, $+=$, $-=$, $*=$, $/=$, $%=$)

→ Arithmetic Operator (+)

The $+$ operator is used for addition in Kotlin. It can be applied to numbers and also for string concatenation.

Ex

```
val a = 10
```

```
val b = 5
```

```
val sum = a + b
```

```
print(sum)
```

15

(D)

1) what is collection? List Kotlin collection explain any one with example

→ collections are a common concept for most programming languages. A collection usually contains a number of objects of the same type and objects in a collection are called element or items. The Kotlin Standard Library provides a comprehensive set of tools for managing collections.

~~The for~~

→ A collection in Kotlin is a data structure used to store, retrieve, and manipulate a group of objects efficiently.



Kotlin provides built-in collection frameworks that include lists, set, and maps. Collections in Kotlin are broadly classified into mutable (modifiable) and immutable (read-only) types.

→ Types :

1) List :- Ordered collection that allows duplicate elements.

→ List is an ordered collection with access to element by indices.

2) Set :- Set is a collection of unique elements which means a group of objects without repetitions.

→ Unordered collection that does not allow duplicate element

3) Map :- A collection of key-value pairs where each key is unique.

⇒ List collection :-

List in Kotlin is an ordered collection that can contain duplicate elements. It provides methods for accessing elements by their indices, iterating the list and performing various operations like filtering, mapping and storing :-

Ex :-

fun main() {

↪

Ex of the Immutable list

```
Val numbers = listof (1, 2, 3, 4, 5)
println (numbers [2]) // output - 3
```

Ex of the Mutable list

```
Val numbers mutablelist = mutablelistof (1, 2, 3)
mutablenumbers.add (4)
print (mutablenumbers)
```

2) Explain Looping controls of Kotlin

1) for loop - The for loop is used to iterate over a range, collection, or Array. It is commonly used when you know the number of iterations in advance.

Ex 1

```
fun main () {
    val numbers = listof (1, 2, 3, 4, 5)
    for (number in numbers)
        println (number)
```

```
    for (i in 1..5) {
        print (i)
    }
```

2) while loop - The while loop is used when the number of iterations is not known in advance, the loop



continues to execute as long as the specified condition is true.

Ex 2

```
fun main() {
```

```
    var i = 1
```

```
    while (i <= 5) {
```

```
        println(i)
```

```
        i++    }
```

```
}
```

3) do-while loop ? The do-while loop is similar to the while loop, but the main difference is that the code block is executed at least once before the condition is checked. The loop continues to execute as long as the specified condition is true.

Ex 3

```
fun main() {
```

```
    var i = 1
```

```
    do {
```

```
        println(i)
```

```
        i++
```

```
    } while (i <= 5)
```

```
}
```


Q-2 A)

- 1) AVD & Android virtual Device
- 2) DVM & Dalvik virtual Machine
- 3) ADB & Android Debug Bridge
- 4) SDK & Software Development Kit

B)

- 1) ~~What~~ Explain Intents.
→ In Android, an intent is a messaging object used to request an action from another app component. Intents facilitate communication between different components of an application such as activities, services, and broadcast ~~send~~ receivers. They can also be used to launch activities, start services or deliver broadcasts.

There are Two types of intents.

- 1) Explicit intent & Directly specifies the target component by its class name. It is commonly used within the same application.
- 2) Implicit intent & Does not specify the target component. Instead, it declares a general action to be performed allowing any Application that can handle the action to respond.



2) Explain Broadcast Receiver.

→ A Broadcast Receiver is an Android component that listens for system-wide or app-specific broadcast messages. It helps application respond to system events like battery changes, network connectivity, or custom intents.

Ex:

```
class MyReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context?,
                           intent: Intent?) {
        Toast.makeText(context, "Broadcast Received!",
            Toast.LENGTH_SHORT).show()
    }
}
```

→ To Register a Broadcast Receiver, you can either declare it in the `AndroidManifest.xml` file or register it dynamically in your code.

(6)

1) Explain Android manifest file.

→ The Android Manifest file (`AndroidManifest.xml`) is a crucial component in every Android application. It serves as a declaration file that provides essential information about the application to the Android system. Here File is:

1) Package Name & Defines the unique identifier for the application.

2) Components Declaration & lists all the application components, including activities, services,



- broadcast receivers, and content providers
- 3) permissions & Specifies the permission the app requires to function, such as internet access, location services and more
 - 4) Intent Filters & Declares the types of intents the app can handle, allowing the app to respond to specific actions or data types.
 - 5) Hardware and Software Features & Specifies the hardware and software features the app needs, such as camera, Bluetooth or OpenGL

2) Explain Services in android

→ A Service is a component that runs in the background to perform long-running operations without needing to interact with the user and it works even if application is destroyed. A service can essentially take two states &

1) Started & A service is started when application component, such as an activity, starts it by calling `startService()`. Once started, a service can run in the background indefinitely, even if the component that started it is destroyed.

2) Bound & A service is bound when an application component binds to it by calling `bindService()`. A bound service



offers a client-server interface that allows components to interact with the services, send requests, get results and even do so across processes with interprocess communication (IPC).

(1)

1) Explain Activity lifecycle with example



→ The Activity lifecycle in Android defines the different states an activity goes through from creation to destruction. the lifecycle is managed by the Android System and consists of several callback methods.

→ Methods :

- 1) onCreate() is called when the activity is created. Used for initialization
- 2) onStart() is called when the activity becomes visible to the user
- 3) onResume() is called when the activity comes to the foreground and is ready for user interaction
- 4) onPause() is called when the activity is partially visible but not in focus
- 5) onStop() is called when the activity is no longer visible to the user
- 6) onRestart() is called when the activity is restarted after being stopped.
- 7) onDestroy() is called before the activity is completely destroyed.

Ex :

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        Log.d("Lifecycle", "onCreate called")
    }
}
```




```
override fun onStart() {
```

```
    super.onStart()
```

```
    log.d("Lifecycle", "onStart called") }
```

```
override fun onResume() {
```

```
    super.onResume()
```

```
    log.d("Lifecycle", "onResume called") }
```

```
override fun onPause() {
```

```
    super.onPause()
```

```
    log.d("Lifecycle", "onPause called") }
```

```
override fun onStop() {
```

```
    super.onStop()
```

```
    log.d("Lifecycle", "onStop called") }
```

```
override fun onRestart() {
```

```
    super.onRestart()
```

```
    log.d("Lifecycle", "onRestart called") }
```

```
override fun onDestroy() {
```

```
    super.onDestroy()
```

```
    log.d("Lifecycle", "onDestroy called") }
```

```
}
```


2) List and explain resources of Android App:
 → In Android, resources and external elements such as images, strings, layouts, and colors that help design and manage the app's UI. These resources are stored in the `res` directory and allow developers to support multiple devices, screen sizes, and languages without modifying the code.

→ Types of Resource & ...

1) Drawable Resources (`res/drawable`)

→ Used for images, shapes, and XML-based graphics.

→ Example: Storing PNG, JPEG, & XML Vector images

Ex: (`Circle.xml`)

```
<shape xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:shape="oval">
```

```
    <solid android:color="#FF0000"/>
```

```
</shape>
```

2) Layout Resource (`res/layout`)

→ Defines the UI structure using XML files.

→ Ex: `activity_main.xml` defines a screen layout.

Ex: `<LinearLayout>`

```
    android:layout_width="match-parent"
```

```
    android:layout_height="match-parent"
```




```

    android:orientation="vertical">
    <TextView> android:text="Hello Android!" />
</LinearLayout>

```

3) String Resources (res/values/strings.xml)

→ stores text values to support multiple languages

Ex:

```
<resources>
```

```
    <string name="app-name">My App</string>
```

```
</resources>
```

• Access in code: `getString(R.string.app-name)`

4) Color Resources (res/values/color.xml)

→ Defines colors for UI components.

Ex:

```
<resources>
```

```
    <color name="white">#FFFFFF</color>
```

```
</resources>
```

• Use in XML: `android:textColor="@color/white"`

5) Dimen Resources (res/values/dimens.xml)

→ Stores size values for uniform spacing and Scaling

Ex:

```
<resource>
```

```
    <dimen name="text-size">16sp</dimen>
```

```
</resource>
```

Use in XML: `android:textSize="@dimen/text-size"`

①-3 A

1) strings.xml File is used to store string resource in Android.

2) TextView widget is used to display static text.

3) textSize property of control determines font size.

4) All activity extends AppCompatActivity class.

(3)

1) Explain types of Animation

1) Property Animations & These animations change the properties of objects, such as position, size, alpha, and rotation. The ObjectAnimator class is commonly used for property animations.

2) Drawable Animations & These animation involve changing the drawable resource of a view. This can be achieved using AnimationDrawable, which is a sequencer of images played one after another.



2) Explain Fragment.

→ In Android Fragment is a part of an activity which enable more modular activity design. It will not be wrong if we say a fragment is a kind of sub-activity. It represents a behaviour or a portion of user interface in an Activity. We always can combine multiple Fragment in single Activity to build a multi-panel UI and reuse a Fragment in multiple Activities. We always need to embed Fragment in activity and fragment lifecycle is directly affected by the host activity's lifecycle. We need to import `android.support.design.widget.Fragment` class in Android application to create Fragment.

3)

1) What Explain Dialogs with Example

→ A Dialog in Android is a small popup window that appears on the screen to interact with the user. It is commonly used for displaying messages, confirming actions or gathering user input.

Types &

1) Alert Dialog & Used for showing alerts with buttons like "OK" and "Cancel"

2) Progress Dialog & Displays loading indicator

3) DatePicker Dialog - Allows the user to pick a date

4) TimePicker Dialog - Allows the user to pick a time



We need to import `android.app.AlertDialog` class in Android application. Following is the code :-

1) Creating dialog Box

```
AlertDialog alert = builder.create();
// Setting the title manually
alert.setTitle("AlertDialog Box");
alert.show();
```

2) Explain Edit Text

→ Android `EditText` is a subclass of `TextView` to use `EditText` in Android application, we need to import `android.widget.EditText` class. `EditText` is used for entering and modifying text. While using `EditText` widget, we must specify its input type in `inputType` property of `EditText` which configures the keyboard according to input. Style of `EditText` shown in the figure.

Box

<EditText

```
android:id="@+id/editText"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:hint="Enter your name"
android:inputType="text"/>
```




1) Explain types of layouts in Android.

1) Relative layout :-

Relative layout is a highly flexible layout used in Android for custom UI design. It allows components to be positioned relative to each other or to the parent container.

This flexibility makes it one of the most commonly used layouts after linear layout. Developers can use attributes like above, below, left and right to arrange elements based on their relationship with other components.

2) Linear layout :-

Linear layout arranges elements in a single direction, either vertically or horizontally. All child elements inside a linear layout follow this specified orientation. It is simple to use, but can lead to performance issues when nesting multiple linear layouts, as deep hierarchies slow down rendering.

3) Table layout :-

Table layout is used to arrange UI components in rows and columns. It does not display visible grid lines but aligns elements based on the row with the most cells, while cells cannot span multiple columns like in HTML tables. Developers can leave some cells empty for alignment purposes.



4) Frame layout: Frame layout is one of the simplest layouts designed to block a specific area of the screen. It is primarily used to hold a single child view, but multiple children can be added, with overlapping views controlled using the `android:layout_gravity` attribute. This layout is often used for fragments, overlays, and modal elements.

2) Explain ListView and GridView.

ListView & Android ListView is a view which contains the group of items and displays in a scrollable list. ListView is implemented by importing `android.widget.ListView` class. ListView is a default scroll view which does not use other scroll views. ListView uses Adapter classes which add the content from data source (such as string array, array, database etc) to ListView. Adapter bridges data between an AdapterView and other views (ListView, ScrollView etc).

Code:

<ListView

`android:id="@+id/list-view"`

`android:layout_width="match_parent"`

`android:layout_height="fill_parent"`

//



Gridview :- in android gridview is a view group that display items in two dimensional scrolling grid (rows and columns). the grid items are not necessarily predetermined but they are automatically inserted to the layout using a ListAdapter. User can then select any grid item by clicking on it. Gridview is default scrollable so we don't need to use ScrollView or anything else with Gridview. Gridview is widely used in android applications. To use Gridview in android application we have to import android.widget.Gridview class.

Code :-

<Gridview

android: id = "@id/simple_Gridview"

android: layout_width = "fill-parent"

android: layout_height = "wrap-content"

android: numColumns = "3" />

①-4 A

1) What is Shared Preference?

Shared Preferences in Android is a lightweight storage mechanism used to store small amounts of key-value pair data persistently. It is mainly used to save user setting, preference, or simple data needs to be retained across app sessions.

2) URI :- Uniform Resource Identifier

3) SQLite data base is used in mobile device to manage data.

4) Small collection key values that you would like to save, you should use the Shared Preferences API.

(B)

1) What is SQLite ?

→ SQLite is a lightweight, embedded, and open-source relational database used in Android to store and manage structured data locally. It does not require a separate database server and allows app to perform (CRUD (Create, Read, Update, Delete) operation efficiently. SQLite is ideal for managing offline data storage in mobile application.

2) What is content provider ?

A Content Provider is a component in Android that allows secure sharing of data between application. It manages access to structured data and enables different apps to read, write and modify data stored in databases, files, or other sources. It uses URI to uniquely identify data and follows a standardized



method for data access, making it useful for sharing contacts, media files and other app data.

(C)

1) Explain SQLiteOpenHelper class

→ For managing all the operations related to the database, a helper class has been given and is called SQLiteOpenHelper. The android.database.sqlite.SQLiteOpenHelper class is used for database creation and version management. For performing any database operation, you have to provide the implementation of onCreate() and onUpgrade() method of SQLiteOpenHelper class. It automatically manages the creation and update of the database. Its syntax is given below

```
public class DBHelper extends SQLiteOpenHelper {
```

```
    public DBHelper() {
```

```
        super(context, DATABASE_NAME, null, 1);
```

```
    public void onCreate(SQLiteDatabase db) { }
```

```
    public void onUpgrade(SQLiteDatabase database,
        int oldVersion, int newVersion) { }
```




Q2) Explain SQLite Database class.

→ The `SQLite.Database` class in Android provides an interface to interact with an SQLite database. It allows performing CRUD (Create, Read, Update, Delete) operations on the database using SQL queries or Android's built-in methods. Key functions include:

1) `execSQL(String sql)` → Executes a single SQL statement.

2) `rawQuery(String sql, String[] selectionArgs)`
Runs a query and returns a result set in a cursor.

3) `insert()`, `update()`, `delete()` → Method to modify database records.

(Q3)

1) Develop an Android application to read and write text file into internal storage.

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">
```

```
<EditText
```

```
    android:id="@+id/editText"
```




```

    android:layout-width="match-parent"
    android:layout-height="wrap-content"
    android:hint="Enter text to save" />

```

<Button

```

    android:id="@+id/btnsave"
    android:layout-width="match-parent"
    android:layout-height="wrap-content"
    android:text="save to File" />

```

<Button

```

    android:id="@+id/btnRead"
    android:layout-width="match-parent"
    android:layout-height="wrap-content"
    android:text="Read from File" />

```

<TextView

```

    android:id="@+id/textView"
    android:layout-width="match-parent"
    android:layout-height="wrap-content"
    android:text="File content will be displayed"
    android:padding="10dp"
    android:textSize="16sp" />

```

</LinearLayout>

Main Activity.kt 2

Package com.example.filehandling

import android.os.Bundle

import android.widget.Button



```
import android.widget.TextView  
import android.widget.Toast  
import androidx.appcompat.app.  
AppCompatActivity  
import java.io.*
```

```
class MainActivity : AppCompatActivity() {
```

```
    private val FILE_NAME = "myfile.txt"
```

```
    var editText : EditText
```

```
    var textView : TextView
```

```
    var btnSave : Button
```

```
    var btnRead : Button
```

```
    override fun onCreate(savedInstanceState :  
        Bundle?) {
```

```
        super.onCreate(savedInstanceState)
```

```
        setContentView(R.layout.activity_main)
```

```
        editText = findViewById(R.id.editText)
```

```
        textView = findViewById(R.id.textView)
```

```
        btnSave = findViewById(R.id.btnSave)
```

```
        btnRead = findViewById(R.id.btnRead)
```

```
        btnSave.setOnClickListener {
```

```
            saveToFile()
```

```
        }
```

```
        btnRead.setOnClickListener {
```

```
            readFromFile()
```

```
        }
```

```
}
```




```
private fun SaveToFile() {  
    val text = editText.text.toString()  
    try {  
        val fos: FileOutputStream = openFileOutput  
            (FILE_NAME, MODE_PRIVATE)  
        fos.write(text.toByteArray())  
        fos.close()  
        Toast.makeText(this, "File saved",  
            Toast.LENGTH_SHORT).show()  
    } catch (e: IOException) {  
        e.printStackTrace()  
    }  
}
```

```
private fun readFromFile() {  
    try {  
        val fis: FileInputStream = openFileInput  
            (FILE_NAME)  
        val data = fis.bufferedReader().use {  
            it.readText()  
        }  
        fis.close()  
        textView.text = data  
    } catch (e: IOException) {  
        e.printStackTrace()  
    }  
}
```


Q-5(A)

1) LBS & Location - Based Services

2) GPS & Global Positioning System

3) JSON & Javascript object Notation

4) What is LocationManager class?

→ LocationManager is a class in Android that provides access to location services, allowing apps to get the device's current location using GPS, network or passive providers.

(B)

1) Explain Notification Manager of Android.

→ Notification Manager in Android is a system service that manages and displays notification in the status bar. It allows app to notify user about events like messages, updates or alerts. Developers use it to create, update, and remove notifications.

Ex:-

```
val notificationManager = getSystemService  
(NOTIFICATION_SERVICE) as  
NotificationManager
```




Q) Explain types of Google Map.

1) Normal Map (MAP-TYPE-NORMAL) ✓

Default map with roads, landmarks, and labels.

2) Satellite Map (MAP-TYPE-SATELLITE) ✓ Displays satellite imagery without road names.

3) Terrain Map ✓ Show elevation and features like mountains.

4) Hybrid Map ✓ Combines satellite imagery with road and label overlays.

(C)

1) Explain telephony API ✓

→ The telephony API is used to, among other things, monitor phone information including the current states of the phone, connections, network etc. In this example, we want to utilize the telephony manager part of the Application Framework and use phone listeners to retrieve various phone states. This is a simple tutorial utilizing the telephony API and its associated listener which can be good before implementing other application functions related to a phone state like connecting the call.

Uses :-

```
Val telephonyManager = getSystemService (TELEPHONY-  
SERVICE) as telephonyManager
```

```
Val imei = telephonyManager.imei
```




Q) Explain web API.

A web API is an online application programming interface that enables developers to interact with external services by using predefined commands to access specific features. It serves as an intuitive interface for retrieving and sending data. For example, a social media platform may provide a web API to fetch user data, while other commonly used APIs support advertising, Machine Learning (ML Kit), and cloud storage. These integrations enhance app functionality, and most successful apps on the play store utilize at least one web API. APIs typically work with XML or JSON to handle large amounts of data efficiently. XML is widely used in Android development, especially for layouts and variable storage, as it structures data using key-value pairs within triangle brackets.

1

1) Develop an Android application to set mark on Google Map

```
import android.os.Bundle;
```

```
import android com.google.android.gms.maps
```

```
CameraUpdateFactory
```

```
import com.google.android.gms.maps;
```

google map



```
import com.google.android.gms.maps.  
    OnMapReadyCallback  
import com.google.android.gms.maps.model.  
    MarkerOptions  
import androidx.appcompat.app.AppCompatActivity  
class MapsActivity : AppCompatActivity(),  
    OnMapReadyCallback {  
  
    private var mMap : GoogleMap  
  
    override fun onCreate(savedInstanceState: Bundle?)  
    {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_maps)  
  
        val mapFragment = SupportFragmentManager.  
            findFragmentById(R.id.map) as SupportMap  
                Fragment  
        mapFragment.getMapAsync(this)  
    }  
  
    override fun onMapReady(googleMap : GoogleMap) {  
        mMap = googleMap  
  
        val location = LatLng(40.7128, -74.0000)  
        mMap.addMarker(MarkerOptions().position(location).  
            title("Murder in New York"))  
        mMap.moveCamera(CameraUpdateFactory.  
            newLatLngZoom(location, 12f))  
    }  
}
```


Q1) Explain Geocoding API

→ The Geocoding API is services provided by Google that converts address like "1600 Amphitheatre Parkway, Mountain View, CA" into geographic coordinates which can be used for mapping and location-based services.

Key Features :-

- Forward Geocoding - Converts a physical address into latitude & longitude
- Reverse Geocoding - Converts latitude & longitude into a human-readable address
- Supports multiple location formats, including street address, place names, and postal codes
- Used in mapping application like Google Maps to pinpoint locations
- Requires an API key from Google Cloud console

Example API Request :-

`https://maps.googleapis.com/maps/api/geocode/json?address=NewYork&key=YOUR-API-KEY`